
Session 25 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rath**

College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

Domain: **AIML (Artificial Intelligence and Machine Learning)**

Github Main Repo Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments

Github File Link :

For Q1-9: https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session25/Question1-9/Assignment%2025%20-%20Q1-9.ipynb

For Q10: https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session25/Question10/Assignment%2025%20-%20Q10.ipynb

Repo created for Deployment :

GitHub Repository: https://github.com/vaishnavi-rathiii/wheat-seed-classification_Streamlit

Deployed Streamlit App: <https://wheat-seed-classificationapp-ufvkh5cfjzeemusdvcsvf.streamlit.app/>

Code & Output

1. Manual Hyperparameter Tuning – KNN

▼

Q1. Manual Hyperparameter Tuning – KNN

```
[109]: import pandas as pd
      from sklearn.datasets import load_iris
      from sklearn.model_selection import train_test_split
      from sklearn.neighbors import KNeighborsClassifier

[110]: iris = load_iris()

      X = iris.data
      y = iris.target

      print("Dataset Loaded Successfully")

      Dataset Loaded Successfully

[111]: X_train, X_test, y_train, y_test = train_test_split(
      X, y, test_size=0.33, random_state=42
      )

      print("Training Samples:", len(X_train))
      print("Testing Samples:", len(X_test))

      Training Samples: 100
      Testing Samples: 50
```

```
[112]: k_values = [3, 5, 7, 11, 13, 15]

scores = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)

    score = model.score(X_test, y_test)
    scores.append(score)
    print(f"K = {k} --> Accuracy = {score:.4f}")
```

```
K = 3 --> Accuracy = 0.9800
K = 5 --> Accuracy = 0.9800
K = 7 --> Accuracy = 0.9800
K = 11 --> Accuracy = 1.0000
K = 13 --> Accuracy = 1.0000
K = 15 --> Accuracy = 1.0000
```

```
[113]: best_score = max(scores)

best_k = k_values[scores.index(best_score)]

print("Best K Value :", best_k)
print("Highest Accuracy :", round(best_score, 4))
```

```
Best K Value : 11
Highest Accuracy : 1.0
```

```
[114]: print("Summary:")
print(f"The highest accuracy was obtained when n_neighbors = {best_k}.")
print("This K value is selected as the optimal hyperparameter for the KNN model.")
```

```
Summary:
The highest accuracy was obtained when n_neighbors = 11.
This K value is selected as the optimal hyperparameter for the KNN model.
```

2. Manual Hyperparameter Tuning – SVM

```
Manual Hyperparameter Tuning – SVM

[115]: from sklearn.svm import SVC

[116]: c_values = [1, 10, 20]
kernels = ["linear", "rbf"]

results = []

for c in c_values:
    for kernel in kernels:
        model = SVC(C=c, kernel=kernel)
        model.fit(X_train, y_train)

        accuracy = model.score(X_test, y_test)
        results.append([c, kernel, accuracy])
        print(f"C = {c}, Kernel = {kernel} --> Accuracy = {accuracy:.4f}")

C = 1, Kernel = linear --> Accuracy = 1.0000
C = 1, Kernel = rbf --> Accuracy = 1.0000
C = 10, Kernel = linear --> Accuracy = 1.0000
C = 10, Kernel = rbf --> Accuracy = 0.9800
C = 20, Kernel = linear --> Accuracy = 0.9800
C = 20, Kernel = rbf --> Accuracy = 1.0000

[117]: comparison = pd.DataFrame(
    results,
    columns=["C", "Kernel", "Accuracy"]
)

print(comparison)

   C  Kernel  Accuracy
0  1  linear    1.00
1  1   rbf     1.00
2 10  linear    1.00
3 10   rbf     0.98
4 20  linear    0.98
5 20   rbf     1.00

[118]: best_result = comparison.loc[
    comparison["Accuracy"].idxmax()
]

print("\nBest Hyperparameters")
print("C :", best_result["C"])
print("Kernel :", best_result["Kernel"])
print("Accuracy :", round(best_result["Accuracy"], 4))

Best Hyperparameters
C : 1
Kernel : linear
Accuracy : 1.0

[119]: print("\nSummary")
print(f"The highest accuracy was achieved using C = {best_result['C']} and Kernel = {best_result['Kernel']}.")
print("This combination is selected as the best hyperparameter setting for the SVM model.")

Summary
The highest accuracy was achieved using C = 1 and Kernel = linear.
This combination is selected as the best hyperparameter setting for the SVM model.
```

3. Grid Search CV

Q3. Grid Search CV

```
[120]: from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

[121]: param_grid = {
        "C": [1, 10, 20],
        "kernel": ["linear", "rbf"]
    }

[122]: svm_grid = GridSearchCV(
        estimator=SVC(),
        param_grid=param_grid,
        cv=5,
        scoring="accuracy"
    )

svm_grid.fit(X_train, y_train)

[122]:
```

GridSearchCV

best_estimator_: SVC

SVC

```
[123]: results = pd.DataFrame(svm_grid.cv_results_)

comparison = results[
    ["param_C", "param_kernel", "mean_test_score"]
]

print(comparison)
```

	param_C	param_kernel	mean_test_score
0	1	linear	0.95
1	1	rbf	0.93
2	10	linear	0.93
3	10	rbf	0.94
4	20	linear	0.93
5	20	rbf	0.95

```
[124]: print("Best Parameters:", svm_grid.best_params_)
print("Best Score:", round(svm_grid.best_score_, 4))

Best Parameters: {'C': 1, 'kernel': 'linear'}
Best Score: 0.95

[125]: print("\nSummary")
print("GridSearchCV tested all combinations of C and kernel using 5-fold cross-validation.")
print("The combination with the highest mean test score was selected as the best model.")

Summary
GridSearchCV tested all combinations of C and kernel using 5-fold cross-validation.
The combination with the highest mean test score was selected as the best model.
```

4. Randomized Search CV

▼

Q4. Randomized Search CV

```
[126]: from sklearn.model_selection import RandomizedSearchCV
       from sklearn.svm import SVC

[127]: param_grid = {
       "C": [1, 10, 20],
       "kernel": ["linear", "rbf"]
       }

[128]: random_search = RandomizedSearchCV(
       estimator=SVC(),
       param_distributions=param_grid,
       n_iter=5,
       cv=5,
       scoring="accuracy",
       random_state=42
       )

random_search.fit(X_train, y_train)
```

[128]:

▶ RandomizedSearchCV ⓘ ?

▶ best_estimator_: SVC

▶ SVC ?

```
[129]: results = pd.DataFrame(random_search.cv_results_)
```

```
comparison = results[
    ["param_C", "param_kernel", "mean_test_score"]
]
```

```
print(comparison)
```

	param_C	param_kernel	mean_test_score
0	1	linear	0.95
1	1	rbf	0.93
2	20	rbf	0.95
3	10	linear	0.93
4	20	linear	0.93

```
[130]: print("Best Parameters:", random_search.best_params_)
print("Best Score:", round(random_search.best_score_, 4))
```

```
Best Parameters: {'kernel': 'linear', 'C': 1}
Best Score: 0.95
```

```
[131]: print("Grid Search Best Parameters :", grid_search.best_params_)
print("Grid Search Best Score :", round(grid_search.best_score_, 4))
print()
print("Random Search Best Parameters :", random_search.best_params_)
print("Random Search Best Score :", round(random_search.best_score_, 4))
```

```
Grid Search Best Parameters : {'max_depth': 3, 'n_estimators': 50}
Grid Search Best Score : 0.9583
```

```
Random Search Best Parameters : {'kernel': 'linear', 'C': 1}
Random Search Best Score : 0.95
```

```
[132]: print("\nSummary")
print("RandomizedSearchCV evaluated 5 randomly selected parameter combinations using 5-fold cross-validation.")
print("GridSearchCV evaluates all possible parameter combinations, whereas RandomizedSearchCV evaluates only a subset, making it faster")
print("The best parameters and scores from both methods are compared above.")
```

Summary
RandomizedSearchCV evaluated 5 randomly selected parameter combinations using 5-fold cross-validation.
GridSearchCV evaluates all possible parameter combinations, whereas RandomizedSearchCV evaluates only a subset, making it faster for larger search spaces.
The best parameters and scores from both methods are compared above.

5. Bagging – Random Forest

```
Q5. Bagging – Random Forest

[133]: from sklearn.ensemble import RandomForestClassifier
       from sklearn.metrics import accuracy_score

[134]: X_train, X_test, y_train, y_test = train_test_split(
       X, y, test_size=0.2, stratify=y, random_state=42
       )

[135]: rf_model = RandomForestClassifier(
       n_estimators=100,
       random_state=42
       )

       rf_model.fit(X_train, y_train)

[135]: ▼ RandomForestClassifier ⓘ ⓘ
       RandomForestClassifier(random_state=42)

[136]: y_pred = rf_model.predict(X_test)

[137]: accuracy = accuracy_score(y_test, y_pred)
       print("Accuracy Score:", round(accuracy, 4))

       Accuracy Score: 0.9

[138]: print("\nSummary")
       print(f"The Random Forest Classifier achieved an accuracy of {accuracy:.4f} on the test dataset.")

       Summary
       The Random Forest Classifier achieved an accuracy of 0.9000 on the test dataset.
```

6. Boosting – AdaBoost & Gradient Boosting

```
Q6. Boosting – AdaBoost & Gradient Boosting

[139]: from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier

[140]: ada_model = AdaBoostClassifier(
       n_estimators=100,
       random_state=42
       )

       ada_model.fit(X_train, y_train)

[140]: ▼ AdaBoostClassifier ⓘ ⓘ
       AdaBoostClassifier(n_estimators=100, random_state=42)

[141]: y_pred_ada = ada_model.predict(X_test)

[142]: ada_accuracy = accuracy_score(y_test, y_pred_ada)
       print("AdaBoost Accuracy:", round(ada_accuracy, 4))

       AdaBoost Accuracy: 0.9333
```



```
[143]: gb_model = GradientBoostingClassifier(  
        n_estimators=100,  
        learning_rate=0.1,  
        random_state=42  
    )  
  
    gb_model.fit(X_train, y_train)
```

```
[143]: ▾ GradientBoostingClassifier ⓘ ?  
      GradientBoostingClassifier(random_state=42)
```

```
[144]: y_pred_gb = gb_model.predict(X_test)
```

```
[145]: gb_accuracy = accuracy_score(y_test, y_pred_gb)  
      print("Gradient Boosting Accuracy:", round(gb_accuracy, 4))  
  
      Gradient Boosting Accuracy: 0.9667
```

```
[146]: print("\nSummary")  
      print(f"AdaBoost Accuracy: {ada_accuracy:.4f}")  
      print(f"Gradient Boosting Accuracy: {gb_accuracy:.4f}")
```

```
Summary  
AdaBoost Accuracy: 0.9333  
Gradient Boosting Accuracy: 0.9667
```

7. Boosting – XGBoost

```
Q7. Boosting – XGBoost

[147]: from xgboost import XGBClassifier

[148]: xgb_model = XGBClassifier(
        n_estimators=100,
        learning_rate=0.1,
        max_depth=3,
        random_state=42
    )

    xgb_model.fit(X_train, y_train)

[148]: XGBClassifier(base_score=None, booster=None, callbacks=None,
        colsample_bylevel=None, colsample_bynode=None,
        colsample_bytree=None, device=None, early_stopping_rounds=None,
        enable_categorical=False, eval_metric=None, feature_types=None,
        feature_weights=None, gamma=None, grow_policy=None,
        importance_type=None, interaction_constraints=None,
        learning_rate=0.1, max_bin=None, max_cat_threshold=None,
        max_cat_to_onehot=None, max_delta_step=None, max_depth=3,
        max_leaves=None, min_child_weight=None, missing=nan,
        monotone_constraints=None, multi_strategy=None, n_estimators=100,
        n_jobs=None, num_parallel_tree=None, ...)

[149]: y_pred_xgb = xgb_model.predict(X_test)

[150]: xgb_accuracy = accuracy_score(y_test, y_pred_xgb)
        print("XGBoost Accuracy:", round(xgb_accuracy, 4))

        XGBoost Accuracy: 0.9333

[151]: print("\nSummary")
        print(f"The XGBoost Classifier achieved an accuracy of {xgb_accuracy:.4f} on the test dataset.")

        Summary
        The XGBoost Classifier achieved an accuracy of 0.9333 on the test dataset.
```

8. Hyperparameter Tuning – Random Forest

Q8. Hyperparameter Tuning on Random Forest

```
[152]: param_grid = {  
        "n_estimators": [50, 100, 150],  
        "max_depth": [3, 5, 7]  
    }
```

```
[153]: rf_grid = GridSearchCV(  
        estimator=RandomForestClassifier(random_state=42),  
        param_grid=param_grid,  
        cv=5,  
        scoring="accuracy"  
    )  
  
rf_grid.fit(X_train, y_train)
```

GridSearchCV

best_estimator_: RandomForestClassifier

RandomForestClassifier

```
[154]: results = pd.DataFrame(rf_grid.cv_results_)  
  
comparison = results[  
    ["param_n_estimators", "param_max_depth", "mean_test_score"]  
]  
  
print(comparison)
```

	param_n_estimators	param_max_depth	mean_test_score
0	50	3	0.958333
1	100	3	0.958333
2	150	3	0.958333
3	50	5	0.950000
4	100	5	0.950000
5	150	5	0.950000
6	50	7	0.950000
7	100	7	0.950000
8	150	7	0.950000

```
[155]: print("Best Parameters:", rf_grid.best_params_)  
print("Best Score:", round(rf_grid.best_score_, 4))  
  
Best Parameters: {'max_depth': 3, 'n_estimators': 50}  
Best Score: 0.9583
```

```
[156]: print("\nSummary")  
print("GridSearchCV evaluated all combinations of n_estimators and max_depth using 5-fold cross-validation.")  
print(f"Best Parameters: {grid_search.best_params_}")  
print(f"Best Score: {grid_search.best_score_:.4f}")
```

Summary
GridSearchCV evaluated all combinations of n_estimators and max_depth using 5-fold cross-validation.
Best Parameters: {'max_depth': 3, 'n_estimators': 50}
Best Score: 0.9583

9. Complete Model Comparison

```
Q9. Complete Model Comparison

[158]: svm_best = SVC(
        C=1,
        kernel="linear"
    )

    svm_best.fit(X_train, y_train)
    y_pred_svm = svm_best.predict(X_test)
    svm_accuracy = accuracy_score(y_test, y_pred_svm)

[159]: y_pred_rf = rf_model.predict(X_test)
    rf_accuracy = accuracy_score(y_test, y_pred_rf)

[160]: y_pred_ada = ada_model.predict(X_test)
    ada_accuracy = accuracy_score(y_test, y_pred_ada)

[161]: y_pred_gb = gb_model.predict(X_test)
    gb_accuracy = accuracy_score(y_test, y_pred_gb)

[162]: y_pred_xgb = xgb_model.predict(X_test)
    xgb_accuracy = accuracy_score(y_test, y_pred_xgb)
```

```
[163]: comparison = pd.DataFrame({
    "Model": [
        "SVM",
        "Random Forest",
        "AdaBoost",
        "Gradient Boosting",
        "XGBoost"
    ],
    "Accuracy Score": [
        svm_accuracy,
        rf_accuracy,
        ada_accuracy,
        gb_accuracy,
        xgb_accuracy
    ]
})

comparison = comparison.sort_values(
    by="Accuracy Score",
    ascending=False
)

comparison.reset_index(drop=True, inplace=True)
print(comparison)
```

	Model	Accuracy Score
0	SVM	1.000000
1	Gradient Boosting	0.966667
2	AdaBoost	0.933333
3	XGBoost	0.933333
4	Random Forest	0.900000

```
[164]: best_model = comparison.iloc[0]

print("\nBest Model :", best_model["Model"])
print("Accuracy Score :", round(best_model["Accuracy Score"], 4))
```

```
Best Model : SVM
Accuracy Score : 1.0
```

```
[165]: print("\nSummary")
print(f"The best performing model is {best_model['Model']} with an accuracy score of {best_model['Accuracy Score']:.4f}.")
```

```
Summary
The best performing model is SVM with an accuracy score of 1.0000.
```

10. Mini Project – Complete Machine Learning Pipeline

i. Import Required Libraries

```
[1]: import pandas as pd
import joblib

from sklearn.model_selection import (
    train_test_split,
    GridSearchCV,
    RandomizedSearchCV
)

from sklearn.svm import SVC
from sklearn.ensemble import (
    RandomForestClassifier,
    AdaBoostClassifier,
    GradientBoostingClassifier
)

from sklearn.metrics import accuracy_score

from xgboost import XGBClassifier
```

ii. Load Dataset & Data Preprocessing & Train-Test Split

```
[6]: # Load Dataset
df = pd.read_csv("seeds.csv")

# Display Dataset
print(df.head())

# Independent Features
X = df.drop("Type", axis=1)

# Dependent Feature
y = df["Type"] - 1

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Dataset Loaded Successfully")
```


	Area	Perimeter	Compactness	Kernel.Length	Kernel.Width	\
0	15.26	14.84	0.8710	5.763	3.312	
1	14.88	14.57	0.8811	5.554	3.333	
2	14.29	14.09	0.9050	5.291	3.337	
3	13.84	13.94	0.8955	5.324	3.379	
4	16.14	14.99	0.9034	5.658	3.562	

	Asymmetry.Coeff	Kernel.Groove	Type
0	2.221	5.220	1
1	1.018	4.956	1
2	2.699	4.825	1
3	2.259	4.805	1
4	1.355	5.175	1

Dataset Loaded Successfully

iii. Manual Hyperparameter Tuning – SVM

```
[3]: # ----- Manual Search -----

c_values = [1, 10, 20]
kernels = ["linear", "rbf"]

best_accuracy = 0
best_c = None
best_kernel = None

print("Manual Search Results\n")

for c in c_values:
    for kernel in kernels:
        svm = SVC(C=c, kernel=kernel)
        svm.fit(X_train, y_train)

        accuracy = svm.score(X_test, y_test)
        print(
            f"C={c}, Kernel={kernel} --> Accuracy={accuracy:.4f}"
        )

        if accuracy > best_accuracy:
            best_accuracy = accuracy
            best_c = c
            best_kernel = kernel
```

```

print("\nBest Parameters")
print("C :", best_c)
print("Kernel :", best_kernel)
print("Accuracy :", round(best_accuracy,4))

```

Manual Search Results

```

C=1, Kernel=linear --> Accuracy=0.8750
C=1, Kernel=rbf --> Accuracy=0.8250
C=10, Kernel=linear --> Accuracy=0.9000
C=10, Kernel=rbf --> Accuracy=0.8500
C=20, Kernel=linear --> Accuracy=0.9000
C=20, Kernel=rbf --> Accuracy=0.8750

```

```

Best Parameters
C : 10
Kernel : linear
Accuracy : 0.9

```

iv. Grid Search CV

```

# ----- Grid Search -----
|
param_grid = {
    "C": [1,10,20],
    "kernel": ["linear", "rbf"]
}

svm_grid = GridSearchCV(
    SVC(),
    param_grid,
    cv=5
)

svm_grid.fit(X_train,y_train)

print("\nGrid Search Best Parameters")
print(svm_grid.best_params_)
print("Best Score :",round(svm_grid.best_score_,4))

```

```

Grid Search Best Parameters
{'C': 10, 'kernel': 'linear'}
Best Score : 0.9623

```

v. Randomized Search CV

```
# ----- Random Search -----  
  
svm_random = RandomizedSearchCV(  
    SVC(),  
    param_grid,  
    n_iter=5,  
    cv=5,  
    random_state=42  
)  
  
svm_random.fit(X_train,y_train)  
  
print("\nRandom Search Best Parameters")  
print(svm_random.best_params_)  
print("Best Score :",round(svm_random.best_score_,4))
```

```
Random Search Best Parameters  
{'kernel': 'linear', 'C': 10}  
Best Score : 0.9623
```

vi. Model Training and Model Comparison and Best Model Selection

```
[8]: svm_model = SVC(  
    C=svm_grid.best_params_["C"],  
    kernel=svm_grid.best_params_["kernel"]  
)  
  
rf_model = RandomForestClassifier(  
    n_estimators=100,  
    random_state=42  
)  
  
ada_model = AdaBoostClassifier(  
    n_estimators=100,  
    random_state=42  
)  
  
gb_model = GradientBoostingClassifier(  
    n_estimators=100,  
    learning_rate=0.1,  
    random_state=42  
)  
  
xgb_model = XGBClassifier(  
    n_estimators=100,  
    learning_rate=0.1,  
    max_depth=3,  
    random_state=42,  
    eval_metric="mlogloss"  
)
```

```

models = {
    "SVM": svm_model,
    "Random Forest": rf_model,
    "AdaBoost": ada_model,
    "Gradient Boosting": gb_model,
    "XGBoost": xgb_model
}

comparison = []

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    accuracy = accuracy_score(y_test, y_pred)
    comparison.append([name, accuracy])

comparison = pd.DataFrame(
    comparison,
    columns=["Model", "Accuracy"]
)

print(comparison)

```

	Model	Accuracy
0	SVM	0.900
1	Random Forest	0.850
2	AdaBoost	0.925
3	Gradient Boosting	0.875
4	XGBoost	0.875

vii. Save Best Model Using Joblib

```

[9]: best_model_name = comparison.loc[
    comparison["Accuracy"].idxmax(),
    "Model"

print("Best Model :", best_model_name)
best_model = models[best_model_name]
joblib.dump(best_model, "best_model.pkl")

print("Best Model Saved Successfully.")

```

```

Best Model : AdaBoost
Best Model Saved Successfully.

```

viii. Streamlit Web Application

The screenshot shows the 'Wheat Seeds Classification System' web application. On the left sidebar, under 'Seeds Classification', there is a 'Project Information' section stating the application predicts wheat seed variety using machine learning. Below this, the 'Dataset' is listed as 'Seeds Dataset (UCI)' and 'Algorithms Trained' includes SVM, Random Forest, AdaBoost, Gradient Boosting, and XGBoost. A 'Built with Streamlit' badge is at the bottom of the sidebar. The main content area has a title 'Wheat Seeds Classification System' and a prompt to 'Enter the measurements of the wheat seed and click Predict.' It features six input fields with sliders: Area (15.0000), Kernel Width (3.3000), Perimeter (14.5000), Asymmetry Coefficient (2.2000), Compactness (0.8700), and Kernel Groove Length (5.0000). A 'Kernel Length' field is also present with a value of 5.5000. A 'Predict Wheat Variety' button is located below the inputs. The output shows 'Predicted Variety: Kama' in a green box.

This screenshot shows the same web application after a prediction. The 'Predicted Variety: Kama' is displayed in a green box. Below this, a section titled 'Wheat Seed Varieties' lists three varieties: Kama, Rosa, and Canadian. Each variety has a list of characteristics: Kama (High grain quality, Good milling performance, Widely cultivated), Rosa (Balanced grain quality, Medium-sized kernels, High productivity), and Canadian (Premium quality wheat, Large kernels, Excellent flour production). The footer of the page reads '© 2026 | Wheat Seeds Classification System | Streamlit + Machine Learning'.